Code Sample

Lynn Yuan

2024-12-04

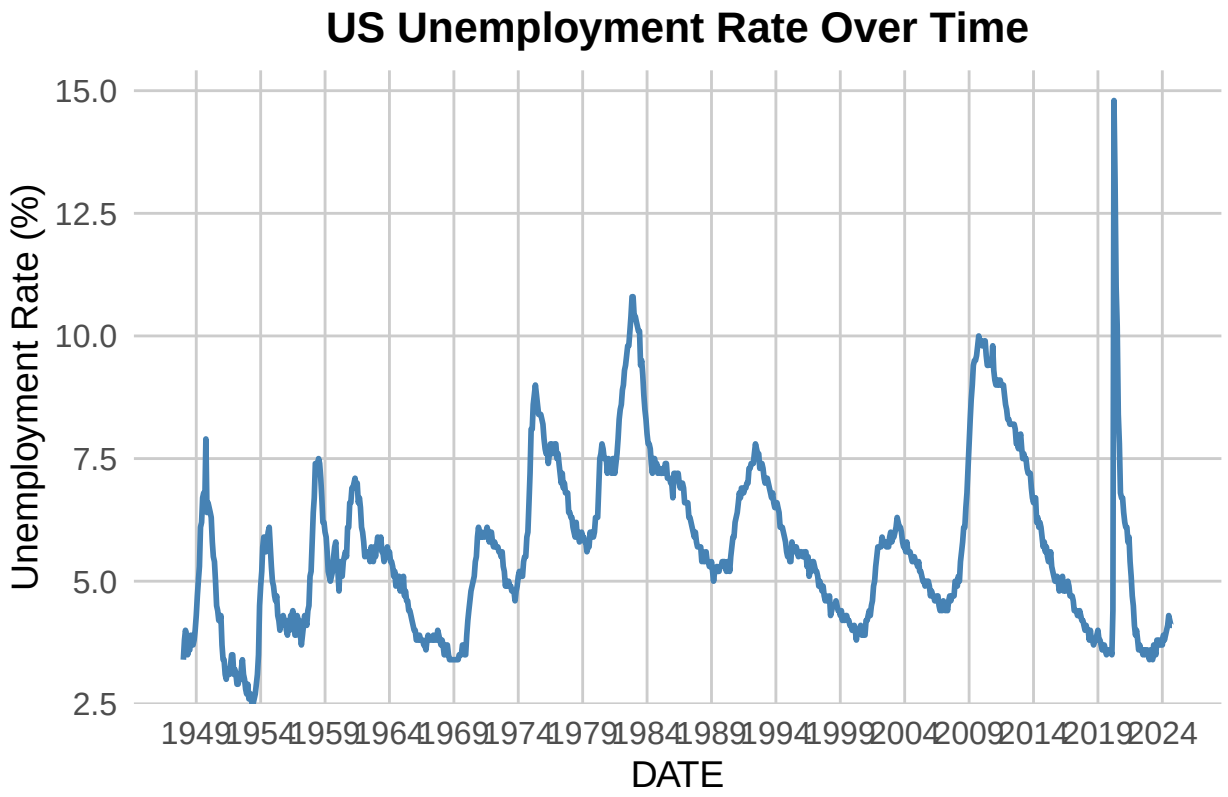
```
library(ggplot2)
library(scales)
library(urca)
library(tseries)
library(mFilter)
library(zoo)
library(gridExtra)
library(forecast)
library(strucchange)
library(TTR)
library(lubridate)
library(tinytex)

# Read data
data <- read.csv("C:\\Users\\blanc\\Downloads\\UNRATE.csv")
data$DATE <- as.Date(data$DATE)
ts_unrate <- ts(data$UNRATE, start = c(1948, 1), frequency = 12)
```

##(Figure 1) US Unemployment Rate Time Series Graph

```
# plot US Unemployment Rate Time Series Graph
ggplot(data, aes(x = DATE, y = UNRATE)) +
  geom_line(color = "steelblue", size = 1) +
  scale_x_date(
    date_labels = "%Y",
    date_breaks = "5 years"
  ) +
  scale_y_continuous(
    name = "Unemployment Rate (%)",
    expand = expansion(mult = c(0, 0.05))
  ) +
  labs(
    title = "US Unemployment Rate Over Time",
    caption = "Source: FRED Database"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12),
    panel.grid.major = element_line(color = "gray80", size = 0.5),
```

```
    panel.grid.minor = element_blank(),
    plot.caption = element_text(hjust = 0, size = 10)
  )
```



Source: FRED Database

#Section 2.1 Augmented Dickey–Fuller test

##(Table 1) Including a time trend in the model:

```
trend_model <- lm(formula = UNRATE ~ as.numeric(DATE), data = data)
summary(trend_model)
```

```
##
## Call:
## lm(formula = UNRATE ~ as.numeric(DATE), data = data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.7894 -1.3270 -0.1947  1.1444  8.7057
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)      5.489e+00  6.916e-02  79.369  < 2e-16 ***
## as.numeric(DATE) 3.298e-05  6.868e-06   4.801 1.84e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 1.69 on 920 degrees of freedom
## Multiple R-squared:  0.02444,    Adjusted R-squared:  0.02338
## F-statistic: 23.05 on 1 and 920 DF,  p-value: 1.84e-06
```

Same lag number (1) for both AIC and BIC

```
# Perform adf test with AIC and BIC
adf_test_aic <- ur.df(ts_unrate, type = "trend", selectlags = "AIC")
adf_test_bic <- ur.df(ts_unrate, type = "trend", selectlags = "BIC")
adf_test_aic@lags
```

```
## [1] 1
```

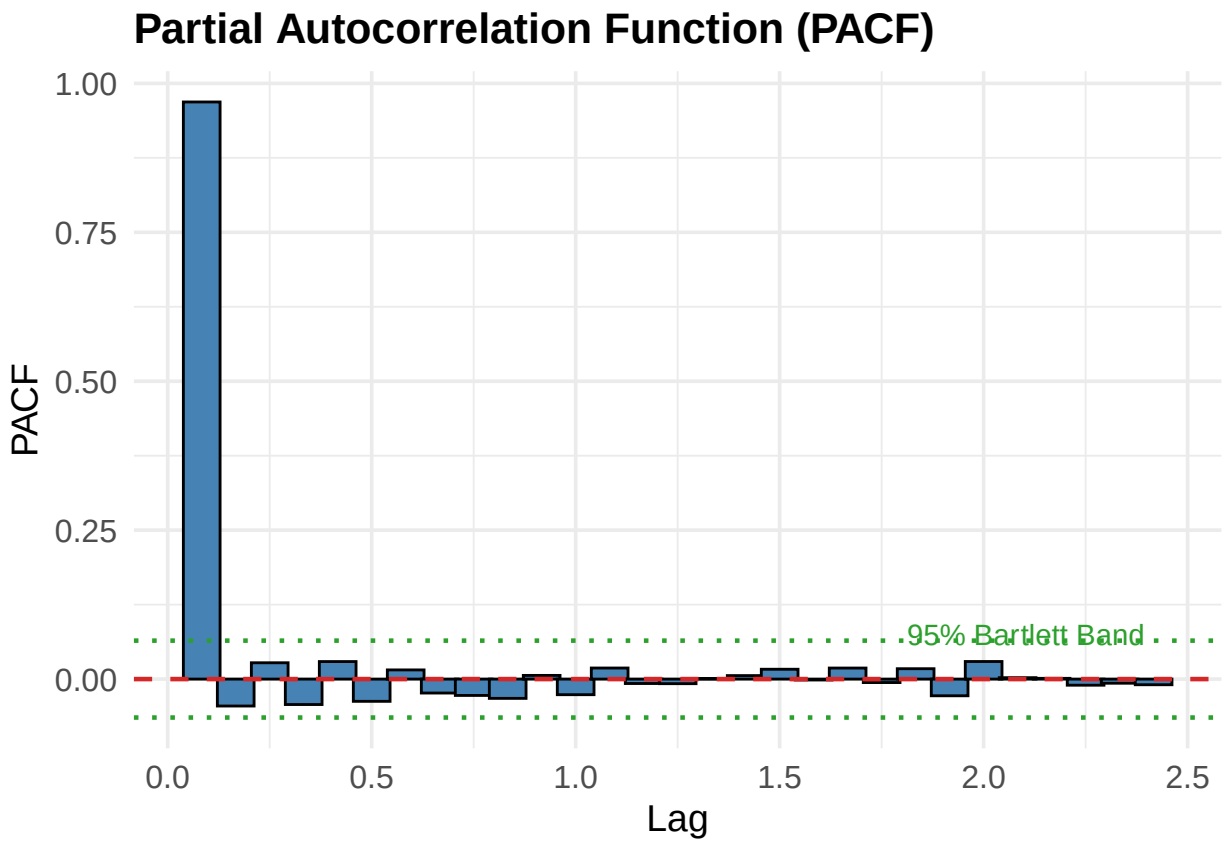
```
adf_test_bic@lags
```

```
## [1] 1
```

##(Figure 2) Partial Autocorrelation Function (PACF)

```
# Compute the Partial Autocorrelation Function (PACF) for the time series
pacf_result <- pacf(ts_unrate, plot = FALSE)
pacf_data <- data.frame(Lag = pacf_result$lag, PACF = pacf_result$acf)
bartlett_ci <- qnorm(0.975) / sqrt(length(ts_unrate))
bar_width <- 0.09

# Plot PACF graph
ggplot(pacf_data, aes(x = Lag, y = PACF)) +
  geom_bar(stat = "identity", fill = "#4682B4", color = "black", width = bar_width) +
  geom_hline(yintercept = 0, color = "#D62728", linetype = "dashed", size = 0.8) +
  geom_hline(yintercept = bartlett_ci, color = "#2ca02c", linetype = "dotted", size = 0.8) +
  geom_hline(yintercept = -bartlett_ci, color = "#2ca02c", linetype = "dotted", size = 0.8) +
  annotate("text", x = max(pacf_data$Lag) * 0.75, y = bartlett_ci + 0.01,
           label = "95% Bartlett Band", color = "#2ca02c", size = 4, hjust = 0) +
  labs(title = "Partial Autocorrelation Function (PACF)",
       x = "Lag",
       y = "PACF") +
  theme_minimal(base_size = 14) +
  theme(axis.text = element_text(size = 12),
        axis.title = element_text(size = 14),
        plot.title = element_text(size = 16, face = "bold"))
```



##(Table 2) ADF test results with a lag of 1 and a trend regression specification.

```
#Perform ADF test
adf_test2 <- ur.df(ts_unrate, type = "trend", lags = 1)
summary(adf_test2)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.8825 -0.1353 -0.0190  0.1034 10.3087
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.772e-01  5.068e-02   3.496 0.000495 ***
## z.lag.1     -3.179e-02  8.166e-03  -3.893 0.000106 ***
## tt           8.697e-06  5.215e-05   0.167 0.867573
```

```
## z.diff.lag   5.168e-02  3.299e-02   1.566 0.117609
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4146 on 916 degrees of freedom
## Multiple R-squared:  0.01777,    Adjusted R-squared:  0.01455
## F-statistic: 5.524 on 3 and 916 DF,  p-value: 0.0009245
##
##
## Value of test-statistic is: -3.8926 5.1217 7.6823
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
```

#Section 2.2 Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test

##(Table 3) KPSS Test Results

```
#Perform KPSS test
kpss_result <- ur.kpss(ts_unrate)
summary(kpss_result)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 6 lags.
##
## Value of test-statistic is: 1.0112
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

#Section 2.3 Zivot-Andrews Test

##(Table 4 & 5) Zivot-Andrews Test Results

```
#Perform zivot-Andrews test
za_test <- ur.za(ts_unrate, model = "both", lag = NULL)
summary(za_test)
```

```
##
## ################################
## # Zivot-Andrews Unit Root Test #
## ################################
##
##
## Call:
```

```
## lm(formula = testmat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.8222 -0.1374 -0.0190  0.1000 10.4068
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.227e-01  5.470e-02   4.072 5.07e-05 ***
## y.l1         9.582e-01  9.037e-03 106.030  < 2e-16 ***
## trend        3.623e-05  7.515e-05   0.482  0.62985
## du           1.579e-01  7.047e-02   2.240  0.02531 *
## dt          -1.631e-03  5.812e-04  -2.807  0.00511 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4135 on 916 degrees of freedom
##   (因为不存在，1个观察量被删除了)
## Multiple R-squared:  0.9417, Adjusted R-squared:  0.9414
## F-statistic:  3699 on 4 and 916 DF,  p-value: < 2.2e-16
##
##
## Teststatistic: -4.6254
## Critical values: 0.01= -5.57 0.05= -5.08 0.1= -4.82
##
## Potential break point at position: 724
```

```
test_statistic <- za_test@teststat
critical_values <- za_test@cval

print(test_statistic)
```

```
## [1] -4.625353
```

```
print(critical_values)
```

```
## [1] -5.57 -5.08 -4.82
```

#Section 2.4 Hodrick–Prescott Filter

##(Figure 3) HP Filter

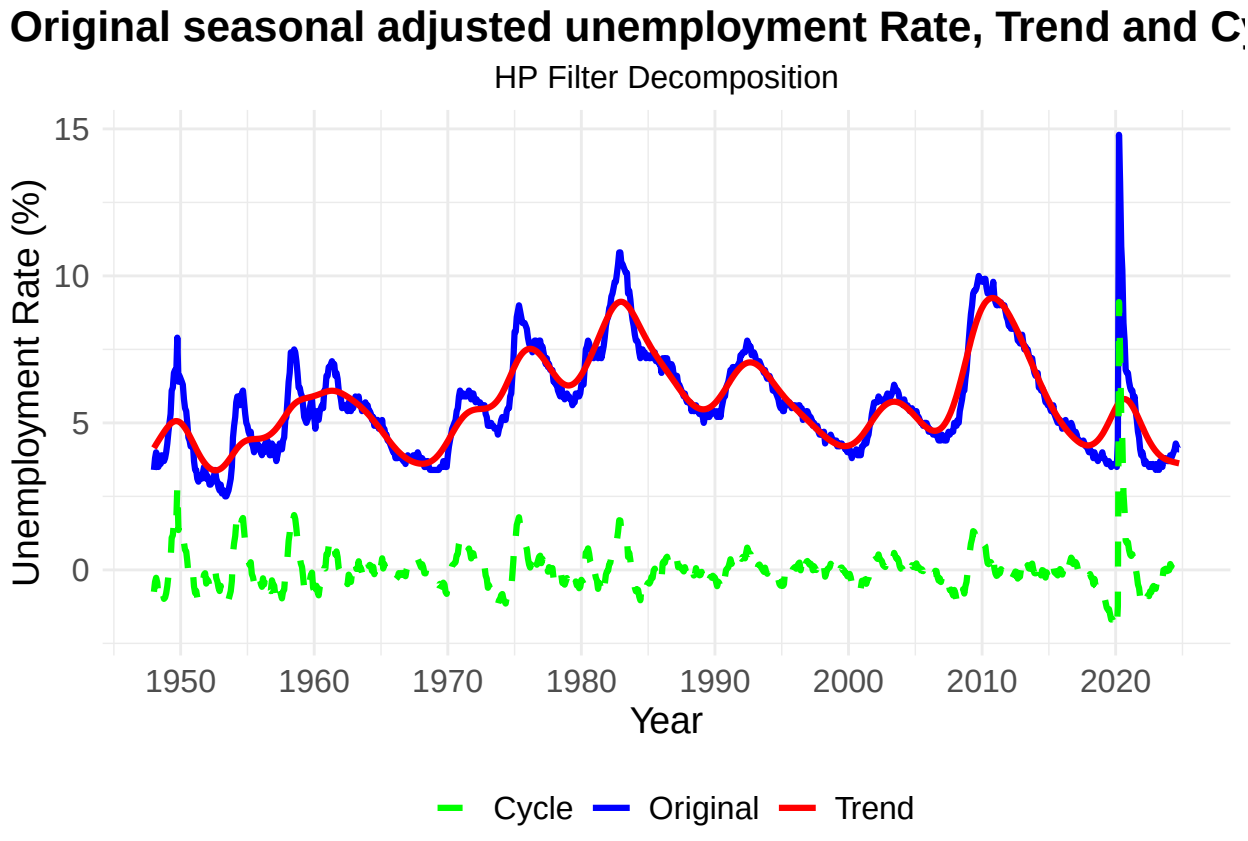
```
hp_result <- hpfilter(ts_unrate, freq = 14400)

#Gain Trend and cycle
trend <- hp_result$trend
cycle <- hp_result$cycle

data_plot <- data.frame(
  Date = data$DATE,
  UNRATE = as.vector(ts_unrate),
  Trend = as.vector(trend),
  Cycle = as.vector(cycle)
```

```
)

# Plot the original, trend, and cycle components
ggplot(data_plot, aes(x = Date)) +
  geom_line(aes(y = UNRATE, color = "Original"), size = 1.1) +
  geom_line(aes(y = Trend, color = "Trend"), size = 1.1, linetype = "solid") +
  geom_line(aes(y = Cycle, color = "Cycle"), size = 1.1, linetype = "dashed") +
  scale_color_manual(values = c("Original" = "blue", "Trend" = "red", "Cycle" = "green")) +
  labs(title = "Original seasonal adjusted unemployment Rate, Trend and Cycle",
       subtitle = "HP Filter Decomposition",
       x = "Year", y = "Unemployment Rate (%)") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 12),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12),
    legend.title = element_text(size = 14),
    legend.text = element_text(size = 12),
    legend.position = "bottom"
  ) +
  guides(color = guide_legend(title = "")) +
  scale_x_date(labels = scales::date_format("%Y"), breaks = scales::pretty_breaks(n = 10))
```



##(Table 6 & 7 & 8 & 9) cycle stationary tests

```r
# ADF Test on Cycle
adf_test_cycle <- ur.df(cycle, type = "drift", selectlags = "BIC")
summary(adf_test_cycle)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + z.diff.lag)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.1358 -0.1327 -0.0168  0.1065 10.0730
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0008985  0.0129856   0.069   0.9448
## z.lag.1     -0.1628576  0.0180430  -9.026   <2e-16 ***
## z.diff.lag   0.0836816  0.0328911   2.544   0.0111 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3939 on 917 degrees of freedom
## Multiple R-squared:  0.0816, Adjusted R-squared:  0.0796
## F-statistic: 40.74 on 2 and 917 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -9.0261 40.7377
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
```

```r
# KPSS Test on Cycle
kpss_test_cycle <- ur.kpss(cycle)
summary(kpss_test_cycle)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 6 lags.
##
## Value of test-statistic is: 0.0094
##
## Critical value for a significance level of:
```

```
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

```
# Zivot-Andrews Test on Cycle (Testing for structural breaks)
za_test_cycle <- ur.za(cycle)
summary(za_test_cycle)
```

```
##
## ################################
## # Zivot-Andrews Unit Root Test #
## ################################
##
##
## Call:
## lm(formula = testmat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.2309 -0.1302 -0.0156  0.1123 10.0488
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.696e-02  2.690e-02   0.630   0.5286
## y.l1         8.441e-01  1.753e-02  48.154   <2e-16 ***
## trend       -5.129e-05  5.378e-05  -0.954   0.3405
## du           1.313e-01  6.030e-02   2.178   0.0296 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3943 on 917 degrees of freedom
##   (因为不存在，1个观察量被删除了)
## Multiple R-squared:  0.7233, Adjusted R-squared:  0.7224
## F-statistic: 799.2 on 3 and 917 DF,  p-value: < 2.2e-16
##
##
## Teststatistic: -8.8911
## Critical values: 0.01= -5.34 0.05= -4.8 0.1= -4.58
##
## Potential break point at position: 866
```

##(Table 10 & 11 & 12 & 13) trend stationary tests

```
# ADF Test on Trend
adf_test_trend <- ur.df(trend, type = "drift", selectlags = "BIC")
summary(adf_test_trend)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
```

```
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + z.diff.lag)
##
## Residuals:
##       Min         1Q     Median         3Q        Max
## -0.0107541 -0.0017119  0.0001688  0.0016918  0.0067542
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.996e-03  4.048e-04   17.28   <2e-16 ***
## z.lag.1     -1.243e-03  6.903e-05  -18.01   <2e-16 ***
## z.diff.lag   9.980e-01  1.910e-03  522.56   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.00297 on 917 degrees of freedom
## Multiple R-squared:  0.9967, Adjusted R-squared:  0.9966
## F-statistic: 1.365e+05 on 2 and 917 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -18.0082 162.4657
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
```

```
# KPSS Test on Trend
kpss_test_trend <- ur.kpss(trend)
summary(kpss_test_trend)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 6 lags.
##
## Value of test-statistic is: 1.3633
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

```
# Zivot-Andrews Test on Trend (Testing for structural breaks)
za_test_trend <- ur.za(trend)
summary(za_test_trend)
```

```
##
## ################################
## # Zivot-Andrews Unit Root Test #
```

```
## ################################
##
##
## Call:
## lm(formula = testmat)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.080911 -0.045069 -0.003443  0.036190  0.143621
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.351e-02  6.955e-03   1.943   0.0523 .
## y.l1         9.973e-01  1.250e-03 797.664  < 2e-16 ***
## trend        1.896e-05  8.553e-06   2.217   0.0269 *
## du          -4.752e-02  6.147e-03  -7.731  2.8e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.04933 on 917 degrees of freedom
##   (因为不存在，1个观察量被删除了)
## Multiple R-squared:  0.9988, Adjusted R-squared:  0.9988
## F-statistic: 2.541e+05 on 3 and 917 DF,  p-value: < 2.2e-16
##
##
## Teststatistic: -2.1377
## Critical values: 0.01= -5.34 0.05= -4.8 0.1= -4.58
##
## Potential break point at position: 774
```

#Section 3.1 ARIMA Model

##(Table 14) BIC values for different segments and Optimal segments' breakdates

```
bp_test <- breakpoints(ts_unrate ~ 1)
summary(bp_test)
```

```
##
##   Optimal (m+1)-segment partition:
##
## Call:
## breakpoints.formula(formula = ts_unrate ~ 1)
##
## Breakpoints at observation number:
##
## m = 1       272
## m = 2       322 467
## m = 3       322 467     784
## m = 4       322 471 646 784
## m = 5   138 322 471 646 784
##
## Corresponding to breakdates:
##
## m = 1           1970(8)
```

```
## m = 2           1974(10) 1986(11)
## m = 3           1974(10) 1986(11)          2013(4)
## m = 4           1974(10) 1987(3)  2001(10) 2013(4)
## m = 5   1959(6) 1974(10) 1987(3)  2001(10) 2013(4)
##
## Fit:
##
## m   0    1    2    3    4    5
## RSS 2692 2291 1916 1804 1704 1689
## BIC 3618 3483 3332 3290 3251 3257
```

##(Figure 4) Mean shifts graph

```r
breakpoints_data <- bp_test$breakpoints
bp_dates <- data$DATE[breakpoints_data]

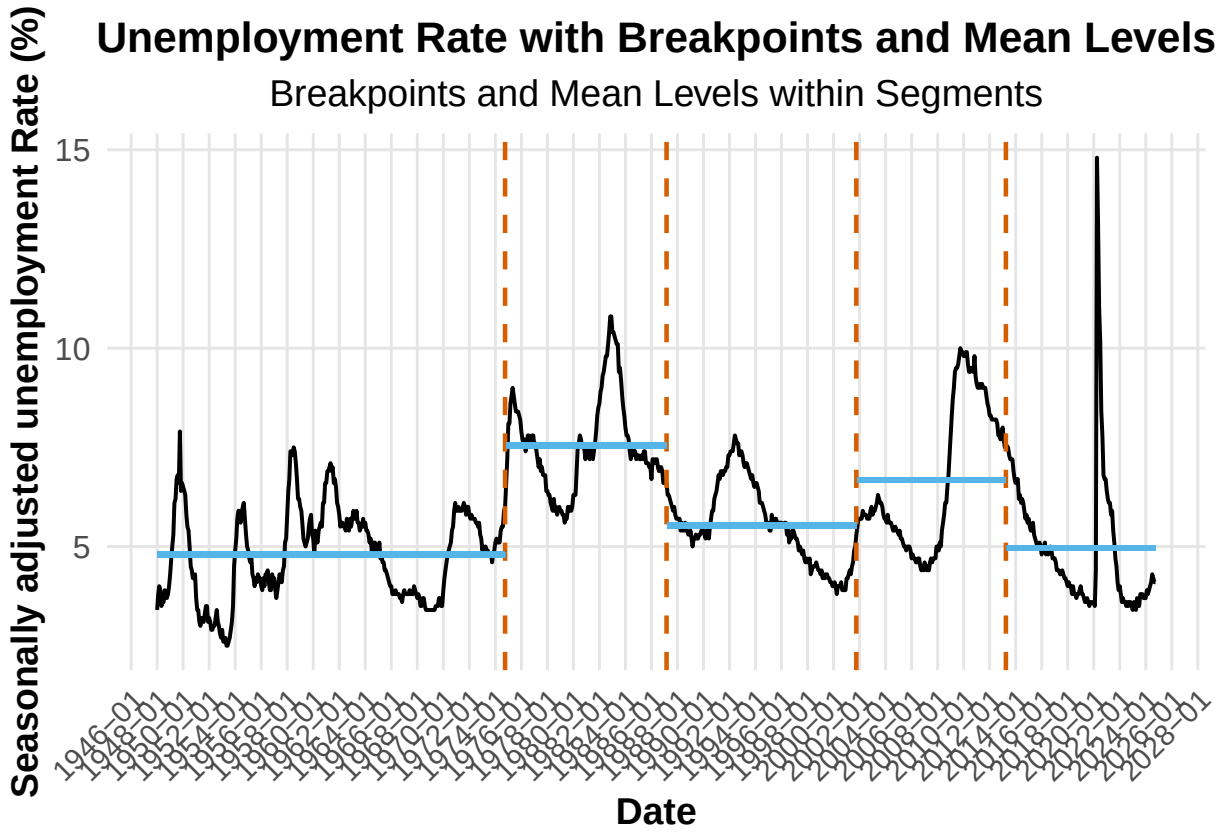
# Calculate mean for each segment
segments_means <- sapply(1:length(breakpoints_data), function(i) {
  if (i == 1) {
    mean(ts_unrate[1:breakpoints_data[i]])
  } else {
    mean(ts_unrate[(breakpoints_data[i-1] + 1):breakpoints_data[i]])
  }
})

# Also calculate mean for the last segment after the last breakpoint
last_mean <- mean(ts_unrate[(breakpoints_data[length(breakpoints_data)] + 1):length(ts_unrate)])
segments_means <- c(segments_means, last_mean)

# Prepare the data for ggplot
plot_data <- data.frame(
  Date = data$DATE,
  UNRATE = ts_unrate
)

# Create the plot
ggplot(plot_data, aes(x = Date, y = UNRATE)) +
  geom_line(color = "black", size = 0.7) +  # Plot the original time series in black
  geom_vline(xintercept = as.numeric(bp_dates), color = "#D55E00", linetype = "dashed", size = 0.8) +
  # Add horizontal lines for means, but only in their corresponding segments
  geom_segment(aes(x = data$DATE[1], y = segments_means[1], xend = data$DATE[breakpoints_data[1]], yend
  sapply(2:length(breakpoints_data), function(i) {
    geom_segment(aes(x = data$DATE[breakpoints_data[i-1] + 1], y = segments_means[i],
                     xend = data$DATE[breakpoints_data[i]], yend = segments_means[i]), color = "#56B4E9
  }) +
  # Add horizontal line for the last segment (after the last breakpoint)
  geom_segment(aes(x = data$DATE[breakpoints_data[length(breakpoints_data)] + 1],
                   y = segments_means[length(segments_means)],
                   xend = data$DATE[length(data$DATE)],
                   yend = segments_means[length(segments_means)]),
               color = "#56B4E9", linetype = "solid", size = 1) +
  labs(
    title = "Unemployment Rate with Breakpoints and Mean Levels",
    x = "Date",
```

```
    y = "Seasonally adjusted unemployment Rate (%)",
    subtitle = "Breakpoints and Mean Levels within Segments"
  ) +
  theme_minimal(base_size = 14) +  # Minimalist theme with larger base font size
  theme(
    panel.grid.major = element_line(color = "gray90", size = 0.5),  # Subtle grid lines
    panel.grid.minor = element_blank(),  # Remove minor grid lines
    axis.title.x = element_text(size = 14, face = "bold"),  # Bold x-axis title
    axis.title.y = element_text(size = 14, face = "bold"),  # Bold y-axis title
    axis.text.x = element_text(angle = 45, hjust = 1),  # Rotate x-axis labels for readability
    axis.text.y = element_text(size = 12),
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),  # Centered and bold title
    plot.subtitle = element_text(hjust = 0.5, size = 14),  # Centered subtitle
    plot.margin = margin(10, 10, 10, 10)  # Add margin for better layout
  ) +
  scale_x_date(labels = date_format("%Y-%m"), breaks = "2 years")  # Format x-axis labels
```



##(Figure 5) ARIMA Forecast of Unemployment Rate

```
last_breakpoint <- breakpoints_data[length(breakpoints_data)]

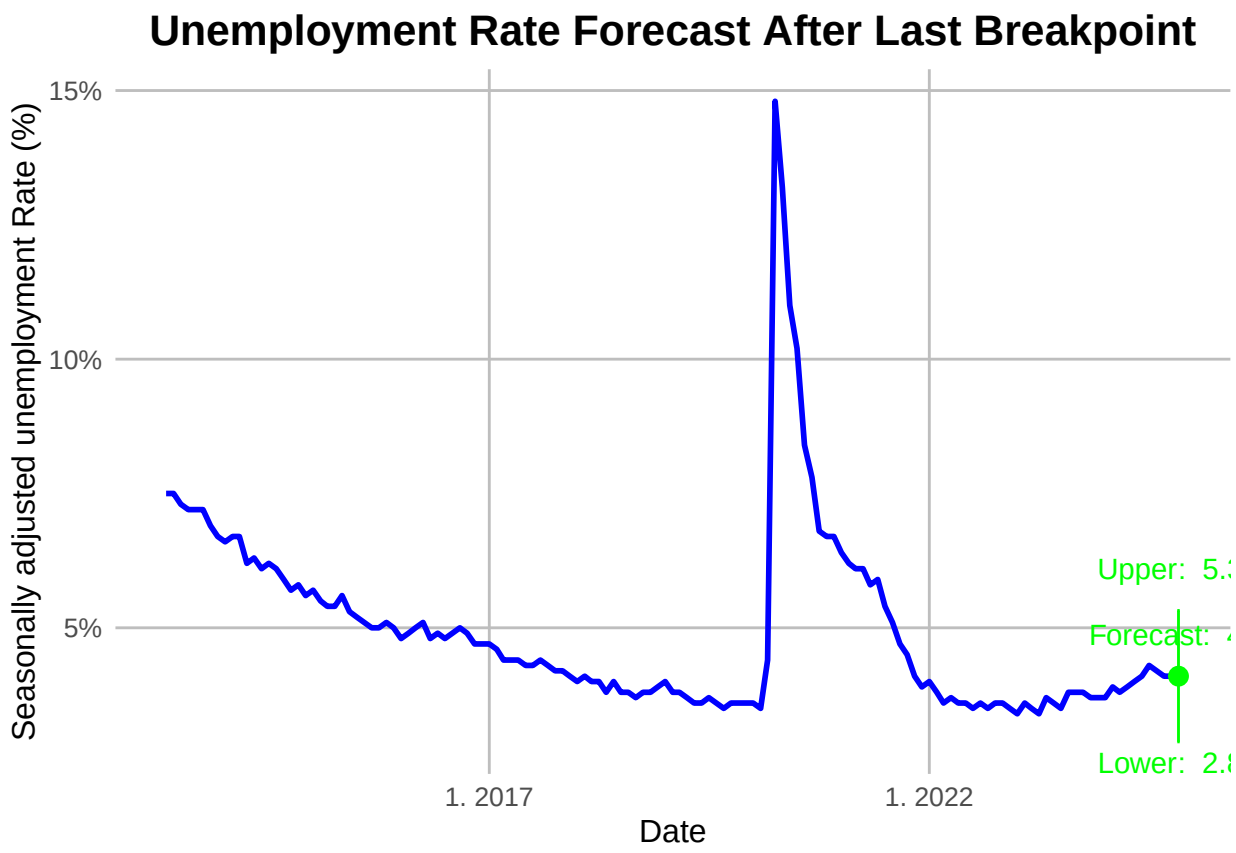
# Extract data after the last breakpoint
ts_after_breakpoint <- window(ts_unrate, start = c(1948, last_breakpoint + 1))

# Fit ARIMA model to the data after the last breakpoint
fit <- auto.arima(ts_after_breakpoint)
```

```r
# Forecast next month's unemployment rate with 95% confidence interval
forecast_result <- forecast(fit, h = 1)

# Get forecasted value and confidence intervals
forecast_value <- forecast_result$mean
lower_bound <- forecast_result$lower[1, 1]
upper_bound <- forecast_result$upper[1, 1]

# Plot
ggplot() +
  geom_line(aes(x = data$DATE[(last_breakpoint + 1):length(data$DATE)],
                y = data$UNRATE[(last_breakpoint + 1):length(data$UNRATE)]),
            color = "blue", size = 1) +
  geom_point(aes(x = max(data$DATE) + 30, y = forecast_value),
             color = "green", size = 3) +
  geom_errorbar(aes(x = max(data$DATE) + 30, ymin = lower_bound, ymax = upper_bound),
                color = "green", width = 10) +
  geom_text(aes(x = max(data$DATE) + 30, y = forecast_value, label = paste("Forecast: ", round(forecast_
            color = "green", vjust = -1.5, hjust = 0.5, size = 4) +
  geom_text(aes(x = max(data$DATE) + 30, y = lower_bound, label = paste("Lower: ", round(lower_bound, 2)
            color = "green", vjust = 1.5, hjust = 0.5, size = 4) +
  geom_text(aes(x = max(data$DATE) + 30, y = upper_bound, label = paste("Upper: ", round(upper_bound, 2)
            color = "green", vjust = -1.5, hjust = 0.5, size = 4) +  # Label for upper bound
  labs(title = "Unemployment Rate Forecast After Last Breakpoint",
       x = "Date", y = "Seasonally adjusted unemployment Rate (%)") +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold", hjust = 0.5),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10),
    panel.grid.major = element_line(color = "gray", size = 0.5),
    panel.grid.minor = element_blank()
  ) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "5 years") +
  scale_y_continuous(labels = scales::percent_format(scale = 1))
```




```
# forecast value and confidence interval
cat("Forecast for next month:", forecast_value, "\n")
```

```
## Forecast for next month: 4.1
```

```
cat("95% confidence interval: [", lower_bound, ",", upper_bound, "]\n")
```

```
## 95% confidence interval: [ 2.877232 , 5.322768 ]
```

#Section 3.2 Time Series Decomposition

##(Figure 6) ACF of Cycle part

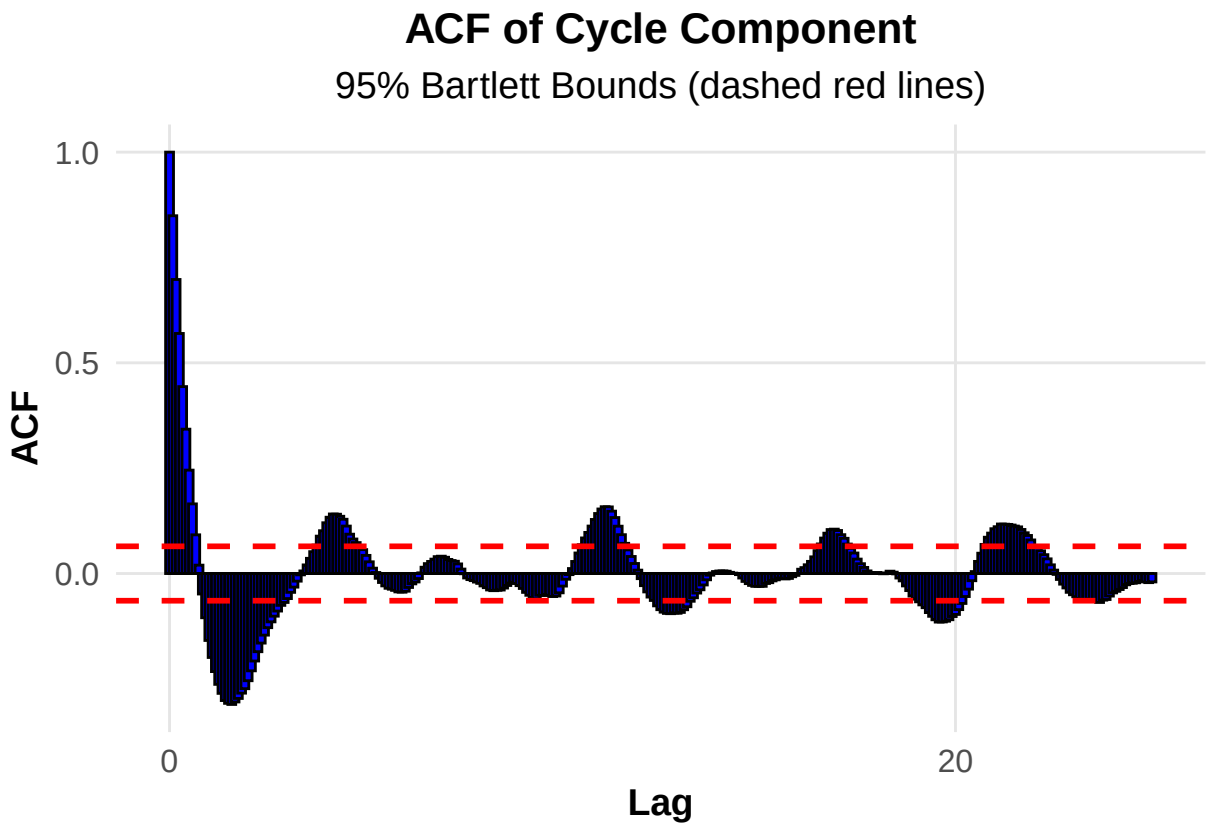
```
acf_result2 <- acf(cycle, plot = FALSE, lag.max = 300)
acf_df2 <- data.frame(Lag = acf_result2$lag, ACF = acf_result2$acf)

# Bartlett Bounds for 95% confidence interval
bartlett_bounds2 <- qnorm(0.975) / sqrt(length(cycle))

# Plot
ggplot(acf_df2, aes(x = Lag, y = ACF)) +

  geom_bar(stat = "identity", fill = "blue", color = "black", width = 0.2) +
  geom_hline(yintercept = bartlett_bounds2, linetype = "dashed", color = "red", size = 1) +
```

```
  geom_hline(yintercept = -bartlett_bounds2, linetype = "dashed", color = "red", size = 1) +
  labs(
    title = "ACF of Cycle Component",
    x = "Lag",
    y = "ACF",
    subtitle = "95% Bartlett Bounds (dashed red lines)"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    panel.grid.major = element_line(color = "gray90", size = 0.5),
    panel.grid.minor = element_blank(),
    axis.title.x = element_text(size = 14, face = "bold"),
    axis.title.y = element_text(size = 14, face = "bold"),
    axis.text.x = element_text(size = 12),
    axis.text.y = element_text(size = 12),
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 14),
    plot.margin = margin(10, 10, 10, 10)
  ) +
  scale_x_continuous(breaks = seq(0, 300, by = 20))
```



##(Figure 7) Harmonic regression

```
# 1. Pre and post break
break_point_2 <- 774
cycle_pre_2 <- cycle[1:break_point_2]
```

```r
cycle_post_2 <- cycle[(break_point_2 + 1):length(cycle)]

# 2. Fourier Analysis and Harmonic Regression for pre-break
fft_pre_2 <- fft(cycle_pre_2)
amplitude_pre_2 <- Mod(fft_pre_2)
frequency_pre_2 <- seq(0, length(cycle_pre_2) - 1) / length(cycle_pre_2)
main_frequency_pre_2 <- frequency_pre_2[which.max(amplitude_pre_2[2:(length(amplitude_pre_2) / 2)])]
main_period_pre_2 <- 1 / main_frequency_pre_2
time_points_pre_2 <- 1:length(cycle_pre_2)
harmonic_terms_pre_2 <- data.frame(
  cos1 = cos(2 * pi * time_points_pre_2 / main_period_pre_2),
  sin1 = sin(2 * pi * time_points_pre_2 / main_period_pre_2)
)
harmonic_model_pre_2 <- lm(cycle_pre_2 ~ cos1 + sin1, data = harmonic_terms_pre_2)

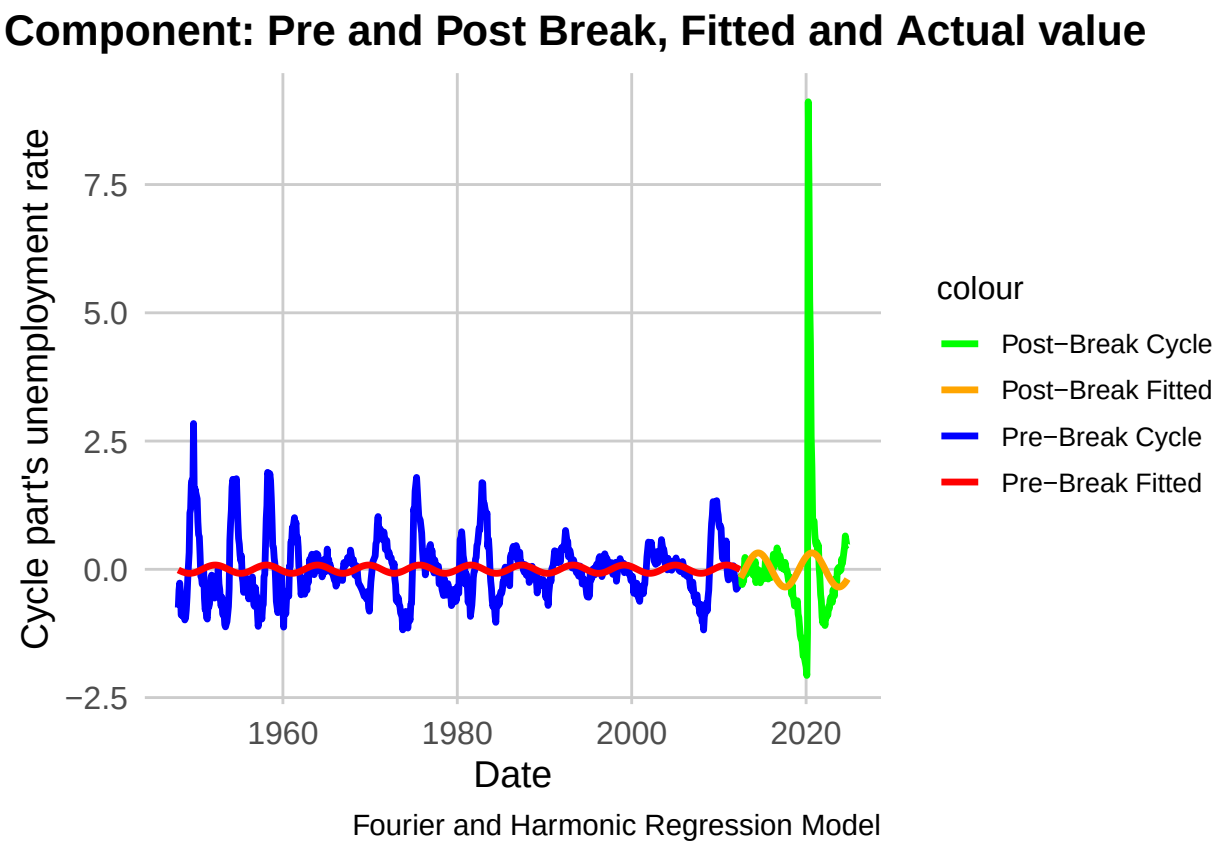
# 3. Fourier Analysis and Harmonic Regression for post-break
fft_post_2 <- fft(cycle_post_2)
amplitude_post_2 <- Mod(fft_post_2)
frequency_post_2 <- seq(0, length(cycle_post_2) - 1) / length(cycle_post_2)
main_frequency_post_2 <- frequency_post_2[which.max(amplitude_post_2[2:(length(amplitude_post_2) / 2)])]
main_period_post_2 <- 1 / main_frequency_post_2
time_points_post_2 <- 1:length(cycle_post_2)
harmonic_terms_post_2 <- data.frame(
  cos1 = cos(2 * pi * time_points_post_2 / main_period_post_2),
  sin1 = sin(2 * pi * time_points_post_2 / main_period_post_2)
)
harmonic_model_post_2 <- lm(cycle_post_2 ~ cos1 + sin1, data = harmonic_terms_post_2)

# 4. Data Frame for Pre-Break Cycle
cycle_pre_df_2 <- data.frame(
  Date = seq.Date(from = as.Date("1948-01-01"), by = "month", length.out = length(cycle_pre_2)),
  Cycle = cycle_pre_2,
  Fitted = fitted(harmonic_model_pre_2)
)

# 5. Data Frame for Post-Break Cycle
cycle_post_df_2 <- data.frame(
  Date = seq.Date(from = as.Date("1948-01-01"), by = "month", length.out = length(cycle_post_2)) + month
  Cycle = cycle_post_2,
  Fitted = fitted(harmonic_model_post_2)
)

# Plot both pre-break and post-break cycle with fitted values
ggplot() +
  geom_line(data = cycle_pre_df_2, aes(x = Date, y = Cycle, color = "Pre-Break Cycle"), size = 1.2) +
  geom_line(data = cycle_pre_df_2, aes(x = Date, y = Fitted, color = "Pre-Break Fitted"), linetype = "s
  geom_line(data = cycle_post_df_2, aes(x = Date, y = Cycle, color = "Post-Break Cycle"), size = 1.2) +
  geom_line(data = cycle_post_df_2, aes(x = Date, y = Fitted, color = "Post-Break Fitted"), linetype =
  labs(title = "Cycle Component: Pre and Post Break, Fitted and Actual value",
       y = "Cycle part's unemployment rate",
       x = "Date",
       caption = "Fourier and Harmonic Regression Model") +
  scale_color_manual(values = c("Pre-Break Cycle" = "blue", "Pre-Break Fitted" = "red", "Post-Break Cycl
```

```
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 12),
    axis.title = element_text(size = 14),
    legend.title = element_text(size = 12),
    legend.text = element_text(size = 10),
    panel.grid.major = element_line(size = 0.5, color = "gray80"),
    panel.grid.minor = element_blank(),
    axis.text = element_text(size = 12)
  )
```



##(Table 15) R-squared values for ARMA models before and after the breakpoint & (Figure 8) ARMA model for cycle

```
# Define breakpoint for splitting the cycle data
breakpoint4<-774

# Split the cycle data into pre-break and post-break segments
cycle_pre4 <- cycle[1:breakpoint4]
cycle_post4 <- cycle[(breakpoint4 + 1):length(cycle)]

# Fit ARIMA models to both pre-break and post-break data
cycle_model_pre4 <- auto.arima(cycle_pre4)
cycle_model_post4 <- auto.arima(cycle_post4)
```

```r
# Forecast the next value for pre-break data
cycle_forecast_pre4 <- forecast(cycle_model_pre4, h = 1)
forecast_values_pre4 <- cycle_forecast_pre4$mean
lower_bound_pre4 <- cycle_forecast_pre4$lower[, 2]
upper_bound_pre4 <- cycle_forecast_pre4$upper[, 2]

# Forecast the next value for post-break data
cycle_forecast_post4 <- forecast(cycle_model_post4, h = 1)
forecast_values_post4 <- cycle_forecast_post4$mean
lower_bound_post4 <- cycle_forecast_post4$lower[, 2]
upper_bound_post4 <- cycle_forecast_post4$upper[, 2]

# Create data frame for pre-break forecast data
forecast_data_pre4 <- data.frame(
  Date = c(tail(data$DATE[1:breakpoint4], 1) + 1),
  Cycle = forecast_values_pre4,
  Lower = lower_bound_pre4,
  Upper = upper_bound_pre4
)

# Create data frame for post-break forecast data
forecast_data_post4 <- data.frame(
  Date = c(tail(data$DATE[(breakpoint4 + 1):length(data$DATE)], 1) + 1),
  Cycle = forecast_values_post4,
  Lower = lower_bound_post4,
  Upper = upper_bound_post4
)

# Extract fitted values (predicted values) for both pre-break and post-break data
fitted_values_pre4 <- fitted(cycle_model_pre4)
fitted_values_post4 <- fitted(cycle_model_post4)

# Create data frames for fitted values
fitted_data_pre4 <- data.frame(Date = data$DATE[1:breakpoint4], Fitted = fitted_values_pre4)
fitted_data_post4 <- data.frame(Date = data$DATE[(breakpoint4 + 1):length(data$DATE)], Fitted = fitted_

# Create data frames for the actual cycle data
cycle_data_pre4 <- data.frame(Date = data$DATE[1:breakpoint4], Cycle = cycle_pre4)
cycle_data_post4 <- data.frame(Date = data$DATE[(breakpoint4 + 1):length(data$DATE)], Cycle = cycle_pos

# Define a function to format dates in year.quarter format
format_date <- function(date) {
  year <- format(date, "%Y")
  quarter <- ceiling(as.numeric(format(date, "%m")) / 3)
  return(paste(year, quarter, sep = "."))
}

breaks_dates <- seq(min(data$DATE), max(data$DATE), by = "5 years")

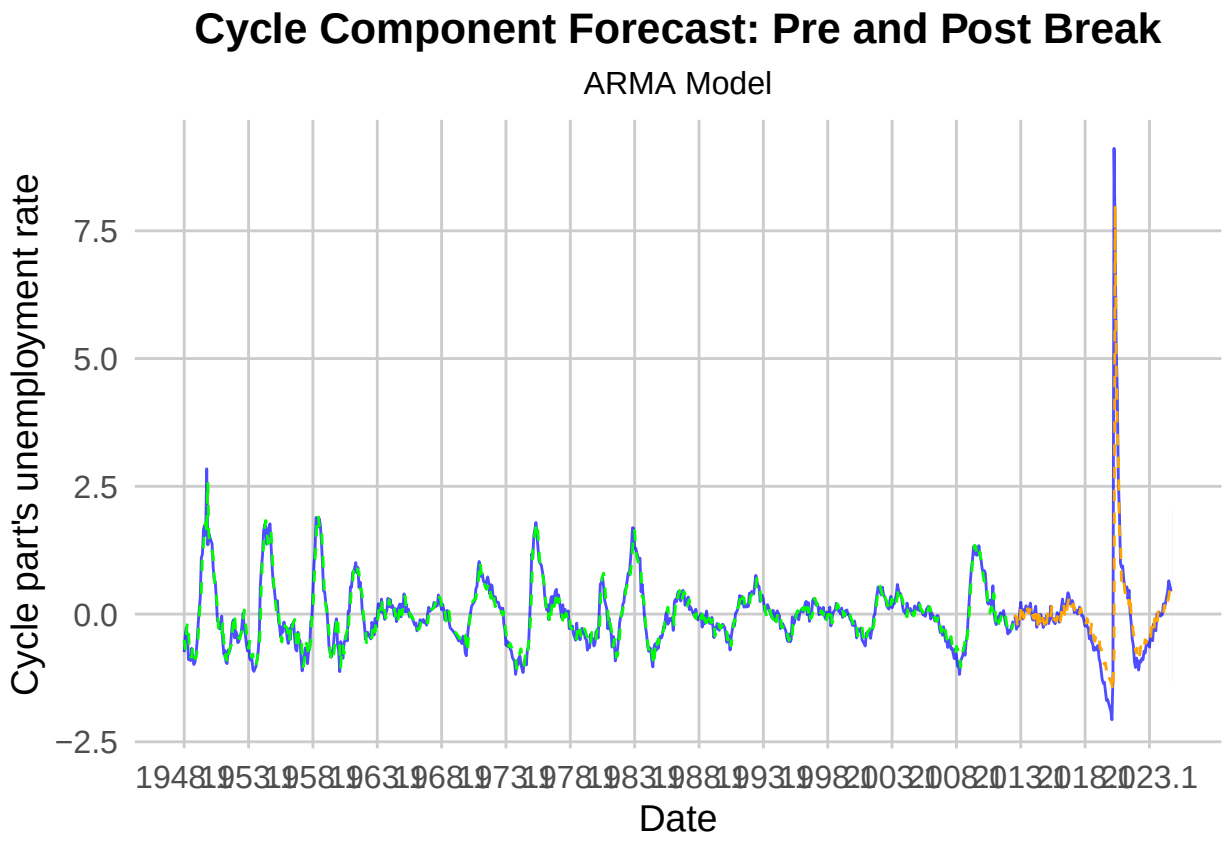
# Create the ggplot to visualize the cycle, fitted values, and forecasted values
ggplot() +
  geom_line(data = cycle_data_pre4, aes(x = Date, y = Cycle), color = "blue", size = 0.5, alpha = 0.7) +
  geom_line(data = fitted_data_pre4, aes(x = Date, y = Fitted), color = "green", linetype = "dashed") +
```

```
  geom_line(data = forecast_data_pre4, aes(x = Date, y = Cycle), color = "red", linetype = "dashed") +
  geom_ribbon(data = forecast_data_pre4, aes(x = Date, ymin = Lower, ymax = Upper), fill = "grey", alpha

  geom_line(data = cycle_data_post4, aes(x = Date, y = Cycle), color = "blue", size = 0.5, alpha = 0.7)
  geom_line(data = fitted_data_post4, aes(x = Date, y = Fitted), color = "orange", linetype = "dashed")
  geom_line(data = forecast_data_post4, aes(x = Date, y = Cycle), color = "red", linetype = "dashed") +
  geom_ribbon(data = forecast_data_post4, aes(x = Date, ymin = Lower, ymax = Upper), fill = "grey", alph

  scale_x_date(
    breaks = breaks_dates,
    labels = format_date(breaks_dates)
  ) +

  labs(title = "Cycle Component Forecast: Pre and Post Break",
       subtitle = "ARMA Model",
       x = "Date",
       y = "Cycle part's unemployment rate") +
  scale_color_manual(values = c("blue", "green", "red")) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 12),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12),
    panel.grid.major = element_line(size = 0.5, color = "gray80"),
    panel.grid.minor = element_blank(),
    legend.position = "none"
  )
```




```
# Calculate R-squared for pre-break ARIMA model
residuals_pre4 <- cycle_pre4 - fitted(cycle_model_pre4)  # Residuals (observed - fitted)
rss_pre4 <- sum(residuals_pre4^2)  # Residual sum of squares
tss_pre4 <- sum((cycle_pre4 - mean(cycle_pre4))^2)  # Total sum of squares
rsq_pre4 <- 1 - rss_pre4 / tss_pre4  # R-squared for pre-break data

# Calculate R-squared for post-break ARIMA model
residuals_post4 <- cycle_post4 - fitted(cycle_model_post4)  # Residuals (observed - fitted)
rss_post4 <- sum(residuals_post4^2)  # Residual sum of squares
tss_post4 <- sum((cycle_post4 - mean(cycle_post4))^2)  # Total sum of squares
rsq_post4 <- 1 - rss_post4 / tss_post4  # R-squared for post-break data

# Print R-squared values for both pre and post-break data
cat("R-squared for pre-break ARIMA model: ", rsq_pre4, "\n")
```

```
## R-squared for pre-break ARIMA model:  0.9003585
```

```
cat("R-squared for post-break ARIMA model: ", rsq_post4, "\n")
```

```
## R-squared for post-break ARIMA model:  0.576845
```

##(Figure 9) ARMA model forecast for cycle

```r
cycle_post <- cycle[775:length(cycle)]

cycle_model_post <- auto.arima(cycle_post)
cycle_forecast_post <- forecast(cycle_model_post, h = 1)

# Extract forecasted values and confidence intervals for the next month
forecast_values_post <- cycle_forecast_post$mean
lower_bound_post <- cycle_forecast_post$lower[, 2]
upper_bound_post <- cycle_forecast_post$upper[, 2]

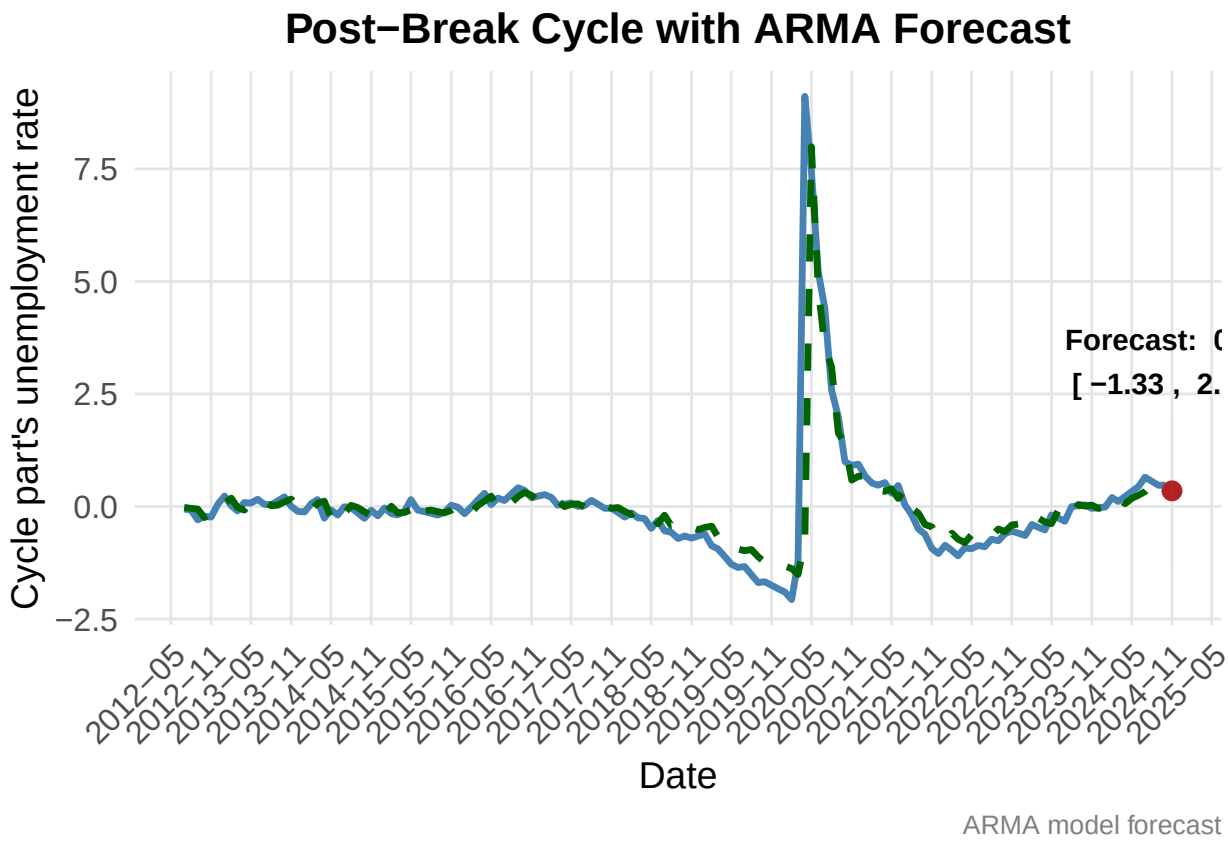
# Get fitted values from the ARIMA model for post-breakpoint data
fitted_values_post <- fitted(cycle_model_post)

# Prepare data for plotting the forecast
forecast_data_post <- data.frame(
  Date = as.Date("2024-11-01"),
  Cycle = forecast_values_post,
  Lower = lower_bound_post,
  Upper = upper_bound_post
)

# Prepare the fitted values data for plotting the post-break cycle
cycle_post_df <- data.frame(
  Date = data$DATE[775:length(data$DATE)],
  Cycle = cycle_post,
  Fitted = fitted_values_post
)

# Plot the post-break cycle with fitted values, forecasted values, and forecast confidence intervals
ggplot(cycle_post_df, aes(x = Date, y = Cycle)) +
  geom_line(color = "steelblue", size = 1.2) +  # Actual cycle values after the breakpoint
  geom_line(aes(y = Fitted), color = "darkgreen", linetype = "dashed", size = 1.2) +  # Fitted values
  geom_point(data = forecast_data_post, aes(x = Date, y = Cycle), color = "firebrick", size = 3) +  # F
  geom_ribbon(data = forecast_data_post, aes(x = Date, ymin = Lower, ymax = Upper), fill = "grey", alpha
  geom_text(data = forecast_data_post, aes(x = Date, y = Cycle, label = paste("Forecast: ", round(Cycle
                                                                              "\n[",round(Lower, 2), ",
            vjust = -1.5, color = "black", size = 4, fontface = "bold") +  # Forecast value with CI lab
  labs(
    title = "Post-Break Cycle with ARMA Forecast",
    x = "Date",
    y = "Cycle part's unemployment rate",
    caption = "ARMA model forecast"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),  # Center the title and make it b
    plot.caption = element_text(hjust = 1, size = 10, color = "gray50"),  # Right-align caption
    axis.text.x = element_text(angle = 45, hjust = 1),
    axis.title = element_text(size = 14),
    axis.text = element_text(size = 12),
    panel.grid.major = element_line(color = "gray90", size = 0.5),
    panel.grid.minor = element_blank(),
    legend.position = "none"
```

```
  ) +
  scale_x_date(labels = scales::date_format("%Y-%m"), breaks = scales::date_breaks("6 months"))
```



##(Figure 10) Polynominal regression for trend

```
# Define time variable as the index of the trend
time <- 1:length(trend)

# Perform breakpoint analysis using quadratic polynomial model
bp_test_quadratic <- breakpoints(trend ~ poly(time, 2))

# Extract breakpoints from the breakpoint analysis
breakpoints <- bp_test_quadratic$breakpoints
# Include the first and last index as breakpoints
breakpoints <- c(1, breakpoints, length(trend))

# Initialize a vector for storing fitted values
fitted_values <- rep(NA, length(trend))

# Define colors for plotting different segments
colors <- c("red", "green", "blue", "purple", "orange")
segments_data <- data.frame()

# Loop through each segment based on breakpoints
for (i in 1:(length(breakpoints) - 1)) {
  segment_start <- breakpoints[i]
```

```
  segment_end <- breakpoints[i + 1]

  # Extract time and trend data for the current segment
  segment_time <- time[segment_start:segment_end]
  segment_trend <- trend[segment_start:segment_end]

  # Initialize a vector to store AIC values for different polynomial degrees
  aic_values <- numeric(26)
  for (degree in 1:26) {
    # Fit polynomial regression models and calculate AIC for each degree
    model <- lm(segment_trend ~ poly(segment_time, degree))
    aic_values[degree] <- AIC(model)
  }

  # Select the polynomial degree that minimizes the AIC
  best_degree <- which.min(aic_values)

  # Fit the best model for the current segment
  best_fit <- lm(segment_trend ~ poly(segment_time, best_degree))
  fitted_values[segment_start:segment_end] <- predict(best_fit, newdata = data.frame(segment_time = segm

  # Assign a color for the current segment
  segment_color <- colors[i %% length(colors) + 1]
  segment_fitted_data <- data.frame(
    Time = segment_time,
    Fitted = fitted_values[segment_start:segment_end],
    Segment = i,
    Color = segment_color
  )

  # Append the segment's fitted data to the overall dataset
  segments_data <- rbind(segments_data, segment_fitted_data)
}

# Process the last segment after the loop
last_segment_start <- breakpoints[length(breakpoints) - 1]
last_segment_end <- breakpoints[length(breakpoints)]
last_segment_time <- time[last_segment_start:last_segment_end]
last_segment_trend <- trend[last_segment_start:last_segment_end]

# Calculate AIC values for the last segment
aic_values_last <- numeric(26)
for (degree in 1:26) {
  model <- lm(last_segment_trend ~ poly(last_segment_time, degree))
  aic_values_last[degree] <- AIC(model)
}
# Select the best polynomial degree for the last segment
best_degree_last <- which.min(aic_values_last)
best_fit_last <- lm(last_segment_trend ~ poly(last_segment_time, best_degree_last))

# Predict the trend for the last segment and calculate confidence intervals
last_segment_fit <- predict(best_fit_last, newdata = data.frame(last_segment_time = last_segment_time),
```

```r
# Predict the trend for the next time point
next_time <- max(last_segment_time) + 1
predicted_trend_next <- predict(best_fit_last, newdata = data.frame(last_segment_time = next_time), int

# Output the predicted trend and confidence interval for the next observation
cat("Predicted trend value for the next observation (time =", next_time, "):", predicted_trend_next[1],
```

```
## Predicted trend value for the next observation (time = 923 ): 3.662842
```

```r
cat("Confidence interval for the next observation:", predicted_trend_next[2], "-", predicted_trend_next
```

```
## Confidence interval for the next observation: 3.638662 - 3.687022
```

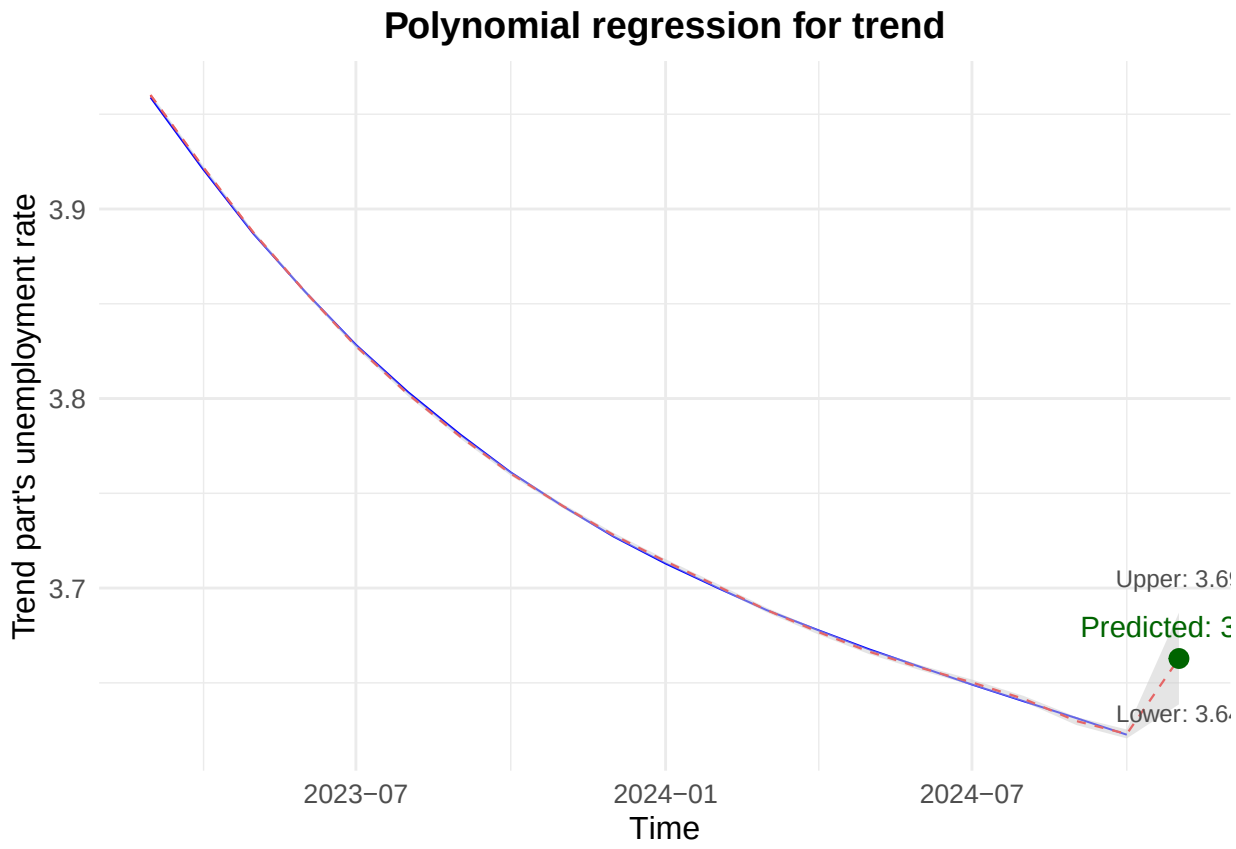
```r
# Generate a sequence of dates for plotting (20 months prior to the last date)
start_date <- as.Date("2024-10-01")
date_seq <- seq.Date(start_date, length.out = 20, by = "-1 month")
date_seq <- rev(date_seq)

# Prepare data for plotting
plot_time <- c(date_seq, as.Date("2024-11-01"))
plot_trend <- c(tail(trend, 20), NA)
plot_fitted <- c(tail(last_segment_fit[, "fit"], 20), predicted_trend_next[1])
plot_lower <- c(tail(last_segment_fit[, "lwr"], 20), predicted_trend_next[2])
plot_upper <- c(tail(last_segment_fit[, "upr"], 20), predicted_trend_next[3])

# Create a data frame for the plot
plot_data <- data.frame(
  Time = plot_time,
  Trend = plot_trend,
  Fitted = plot_fitted,
  Lower = plot_lower,
  Upper = plot_upper
)

# Generate the plot
ggplot() +
  geom_line(data = plot_data, aes(x = Time, y = Trend), color = "blue", linewidth = 0.35, linetype = "s
  geom_line(data = plot_data, aes(x = Time, y = Fitted), color = "red", linetype = "dashed", linewidth =
  geom_ribbon(data = plot_data, aes(x = Time, ymin = Lower, ymax = Upper), fill = "gray80", alpha = 0.5)
  geom_point(aes(x = as.Date("2024-11-01"), y = predicted_trend_next[1]), color = "darkgreen", size = 3)
  geom_text(aes(x = as.Date("2024-11-01"), y = predicted_trend_next[1], label = paste0("Predicted: ", r
            vjust = -1, hjust = 0.5, color = "darkgreen", size = 4) +  # Predicted value label
  geom_text(aes(x = as.Date("2024-11-01"), y = predicted_trend_next[2], label = paste0("Lower: ", round
            vjust = 1, hjust = 0.5, color = "gray30", size = 3) +  # Lower bound label
  geom_text(aes(x = as.Date("2024-11-01"), y = predicted_trend_next[3], label = paste0("Upper: ", round
            vjust = -1.5, hjust = 0.5, color = "gray30", size = 3) +  # Upper bound label
  labs(title = "Polynomial regression for trend",
       x = "Time",
       y = "Trend part's unemployment rate") +
  theme_minimal() +  # Minimal theme
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
```

```
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10)
  )
```



#Section 4.1 Forecast Accuracy Evaluation

##(Table 16) ARIMA Performance & (Table 17) Decomposition Performance

```
# Time decomposition
fitted_pre4 <- fitted(cycle_model_pre4)
fitted_post4 <- fitted(cycle_model_post4)
all_actuals <- c(cycle_pre4, cycle_post4)
all_fitted <- c(fitted_pre4, fitted_post4)
trend_data <- data.frame(Time = time, Trend = trend, Fitted = fitted_values)
actual <- trend_data$Series.1 + all_actuals
fitted <- trend_data$Fitted + all_fitted

# ARIMA
fitted_values1 <- numeric(length(ts_unrate)) * NA
forecast_horizon <- 1
forecast_results <- list()

# Fit ARIMA models for each segment and collect fitted values
for (i in 1:(length(breakpoints_data) + 1)) {
  # Define the segment
  if (i == 1) {
```

```
    segment_indices <- 1:breakpoints_data[i]
  } else if (i == length(breakpoints_data) + 1) {
    segment_indices <- (breakpoints_data[i - 1] + 1):length(ts_unrate)
  } else {
    segment_indices <- (breakpoints_data[i - 1] + 1):breakpoints_data[i]
  }

  segment_ts <- ts_unrate[segment_indices]

  # Fit ARIMA model
  arima_model <- auto.arima(segment_ts)

  # Store fitted values
  fitted_values1[segment_indices] <- fitted(arima_model)

  # For the last segment, store forecast
  if (i == length(breakpoints_data) + 1) {
    forecast_results <- forecast(arima_model, h = forecast_horizon)
  }
}
observed_values <- ts_unrate  # The original time series data
fitted_values1 <- fitted_values1

# R-squared
rss1 <- sum((observed_values - fitted_values1)^2)  # Residual Sum of Squares
tss1 <- sum((observed_values - mean(observed_values))^2)  # Total Sum of Squares
r_squared1 <- 1 - (rss1 / tss1)

# RMSE (Root Mean Squared Error)
rmse1 <- sqrt(mean((observed_values - fitted_values1)^2))

# MSPE (Mean Squared Prediction Error)
mspe1 <- mean((observed_values - fitted_values1)^2)

# MAE (Mean Absolute Error)
mae1 <- mean(abs(observed_values - fitted_values1))

# MAPE (Mean Absolute Percentage Error)
mape1 <- mean(abs((observed_values - fitted_values1) / observed_values)) * 100

# Results
cat("R-squared1: ", r_squared1, "\n")
```

```
## R-squared1:  0.9426569
```

```
cat("RMSE1: ", rmse1, "\n")
```

```
## RMSE1:  0.4091759
```

```
cat("MSPE1: ", mspe1, "\n")
```

```
## MSPE1:  0.1674249
```

```r
cat("MAE1: ", mae1, "\n")
```

```
## MAE1:  0.1575768
```

```r
cat("MAPE1: ", mape1, "%\n")
```

```
## MAPE1:  2.767416 %
```

```r
#Decomposition Performance
# R-squared
rss2 <- sum((actual - fitted)^2)  # Residual Sum of Squares
tss2 <- sum((actual - mean(actual))^2)  # Total Sum of Squares
r_squared2 <- 1 - (rss2 / tss2)

# RMSE (Root Mean Squared Error)
rmse2 <- sqrt(mean((actual - fitted)^2))

# MSPE (Mean Squared Prediction Error)
mspe2 <- mean((actual - fitted)^2)

# MAE (Mean Absolute Error)
mae2 <- mean(abs(actual - fitted))

# MAPE (Mean Absolute Percentage Error)
mape2 <- mean(abs((actual - fitted) / actual)) * 100

# Results
cat("R-squared2: ", r_squared2, "\n")
```

```
## R-squared2:  0.9506904
```

```r
cat("RMSE2: ", rmse2, "\n")
```

```
## RMSE2:  0.3794333
```

```r
cat("MSPE2: ", mspe2, "\n")
```

```
## MSPE2:  0.1439696
```

```r
cat("MAE2: ", mae2, "\n")
```

```
## MAE2:  0.1541752
```

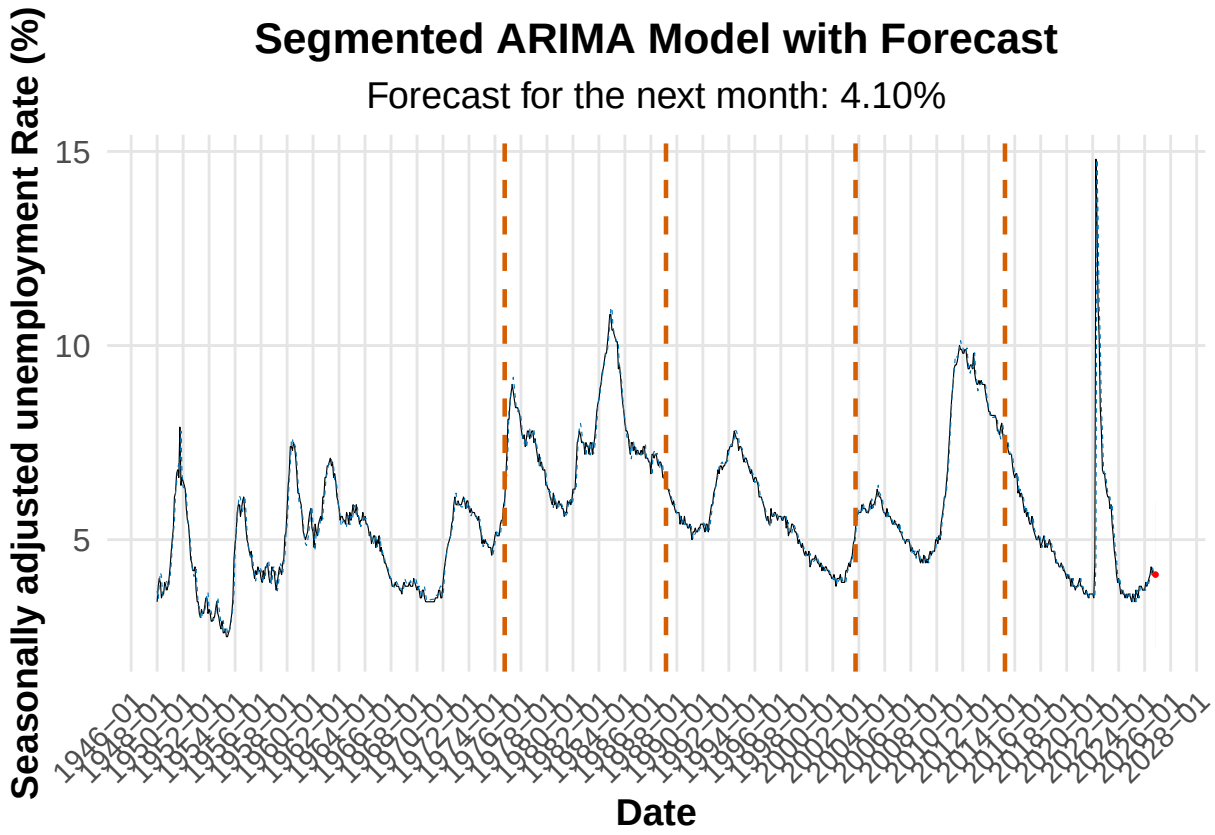
```r
cat("MAPE2: ", mape2, "%\n")
```

```
## MAPE2:  2.831719 %
```

##(Figure 11) ARIMA Forecast of Unemployment Rate

```r
forecast_dates <- seq(max(data$DATE), by = "month", length.out = forecast_horizon + 1)[-1]
 forecast_data <- data.frame(
    Date = forecast_dates,
    Forecast = forecast_results$mean,
    Lower = forecast_results$lower[, 2],
    Upper = forecast_results$upper[, 2]
   )

   # Prepare data for plotting
   plot_data <- data.frame(
      Date = data$DATE,
      UNRATE = ts_unrate,
      Fitted = fitted_values1
     )
 # Plot original series, fitted values, and forecast
   ggplot() +
    # Original time series
     geom_line(data = plot_data, aes(x = Date, y = UNRATE), color = "black", size = 0.2) +
    # Fitted values
     geom_line(data = plot_data, aes(x = Date, y = Fitted), color = "#0072B2", size = 0.2, linetype = "
     # Breakpoints
     geom_vline(xintercept = as.numeric(bp_dates), color = "#D55E00", linetype = "dashed", size = 0.8) +
    # Forecast ribbon
     geom_ribbon(data = forecast_data, aes(x = Date, ymin = Lower, ymax = Upper), fill = "lightblue", al
    # Forecast line and point
     geom_line(data = forecast_data, aes(x = Date, y = Forecast), color = "red", size = 0.2) +
     geom_point(data = forecast_data, aes(x = Date, y = Forecast), color = "red", size = 0.5) +
     # Labels and themes
     labs(
         title = "Segmented ARIMA Model with Forecast",
         x = "Date",
         y = "Seasonally adjusted unemployment Rate (%)",
         subtitle = sprintf("Forecast for the next month: %.2f%%",
                                                  forecast_results$mean,
                                                  forecast_results$lower[2],
                                                  forecast_results$upper[2])
       ) +
     theme_minimal(base_size = 14) +
     theme(
         panel.grid.major = element_line(color = "gray90", size = 0.5),
        panel.grid.minor = element_blank(),
         axis.title.x = element_text(size = 14, face = "bold"),
         axis.title.y = element_text(size = 14, face = "bold"),
         axis.text.x = element_text(angle = 45, hjust = 1),
         axis.text.y = element_text(size = 12),
         plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
         plot.subtitle = element_text(hjust = 0.5, size = 14),
         plot.margin = margin(10, 10, 10, 10)
       ) +
     scale_x_date(labels = date_format("%Y-%m"), breaks = "2 years")
```



#Section 4.2 Forecast Accuracy Tests

##(Table 18) Results of MSPE-based F Test

```
#MSPE (Mean Squared Prediction Error) for both models
mspe_ARIMA <- mean((observed_values - fitted_values1)^2)
mspe_Decomposition <- mean((actual - fitted)^2)


# MSPE-based F Test
f_statistic <- mspe_Decomposition / mspe_ARIMA
p_value_f_test <- pf(f_statistic, df1 = length(observed_values)-1, df2 = length(observed_values)-1, lowe

cat("MSPE-based F Test Statistic:", f_statistic, "\n")
```

```
## MSPE-based F Test Statistic: 0.8599056
```

```
cat("P-value for F Test:", p_value_f_test, "\n")
```

```
## P-value for F Test: 0.9889471
```

##(Table 19) Granger-Newbold Test Results

```r
# Step 1: Calculate the forecast errors
error_model <- observed_values - fitted_values1  # Errors from ARIMA model
error_benchmark <- actual - fitted  # Errors from time decomposition model

x <- error_benchmark+error_model
z <- error_benchmark-error_model

# Step 2: Granger-Newbold Test

# Calculate the sample correlation coefficient r_xz between the errors
r_xz <- cor(x, z)

# Calculate the Granger-Newbold Test statistic
N <- length(error_model)  # The number of forecasts
GN <- r_xz / sqrt((1 - r_xz^2) / (N - 1))

# Calculate the p-value for Granger-Newbold Test (using t-distribution)
p_value_GN <- 2 * (1 - pt(abs(GN), df = N - 1))

cat("Granger-Newbold Test Statistic:", GN, "\n")
```

```
## Granger-Newbold Test Statistic: -5.931314
```

```r
cat("P-value for Granger-Newbold Test:", p_value_GN, "\n")
```

```
## P-value for Granger-Newbold Test: 4.248253e-09
```

##(Table 20) Diebold-Mariano Test Results

```r
library(multDM)

fitted1 <- as.numeric(fitted)
fitted_values2 <- as.numeric(fitted_values1)
ts <- as.numeric(actual)

DM.test(fitted1,fitted_values2,y=ts,loss="SE",h=1,c=FALSE,H1="same")
```

```
##
##  Diebold-Mariano test
##
## data:  fitted1 and fitted_values2 and ts
## statistic = -1.7693, forecast horizon = 1, p-value = 0.07685
## alternative hypothesis: Forecast f1 and f2 have different accuracy.
```

#Section 4.3 Long Term Prediction

##(Figure 12) Long term polynomial prediction

```r
# Generate future time points for the next 12 months
future_time <- (max(last_segment_time) + 1):(max(last_segment_time) + 12)
```

```r
# Predict the future trend values and their confidence intervals using the best model
future_pred <- predict(best_fit_last, newdata = data.frame(last_segment_time = future_time), interval =

# Generate a sequence of dates for the next 12 months
next_years <- seq.Date(from = as.Date("2024-11-01"), by = "month", length.out = 12)

# Combine the previous dates with the future dates for plotting
plot_time2 <- c(date_seq, next_years)

# Extract the fitted values, lower bound, and upper bound for the future predictions
future_fitted_values <- future_pred[, 1]
future_lower_bound <- future_pred[, 2]
future_upper_bound <- future_pred[, 3]

# Prepare the trend and fitted values for plotting, including future values
plot_trend2 <- c(tail(trend, 20), rep(NA, 12))
plot_fitted2 <- c(tail(last_segment_fit[, "fit"], 20), future_fitted_values)

# Prepare the lower and upper bounds for the future predictions
plot_lower2 <- c(tail(last_segment_fit[, "lwr"], 20), future_lower_bound)
plot_upper2 <- c(tail(last_segment_fit[, "upr"], 20), future_upper_bound)

# Create a data frame for the plot with trend, fitted values, and bounds
plot_data <- data.frame(
  Time = plot_time2,
  Trend = plot_trend2,
  Fitted = plot_fitted2,
  Lower = plot_lower2,
  Upper = plot_upper2
)

# Add a column to distinguish the last 12 fitted values (future predictions)
plot_data$Fitted_Color <- ifelse(seq_along(plot_data$Fitted) > (nrow(plot_data) - 12), "red", "blue")

# Generate the plot
ggplot() +
  # Plot the original trend as an orange line
  geom_line(data = plot_data, aes(x = Time, y = Trend), color = "orange", linewidth = 0.5, linetype = "

  # Plot the fitted values with dashed lines and different colors for the past and future
  geom_line(data = plot_data, aes(x = Time, y = Fitted, color = Fitted_Color), linetype = "dashed", lin

  # Set the color scale for fitted values (blue for past, red for future)
  scale_color_manual(values = c("blue", "red")) +

  # Plot the confidence interval as a shaded ribbon
  geom_ribbon(data = plot_data, aes(x = Time, ymin = Lower, ymax = Upper), fill = "gray80", alpha = 0.5

  # Plot the predicted points for the future trend
  geom_point(data = plot_data, aes(x = Time, y = Fitted), color = "darkgreen", size = 0.5) +

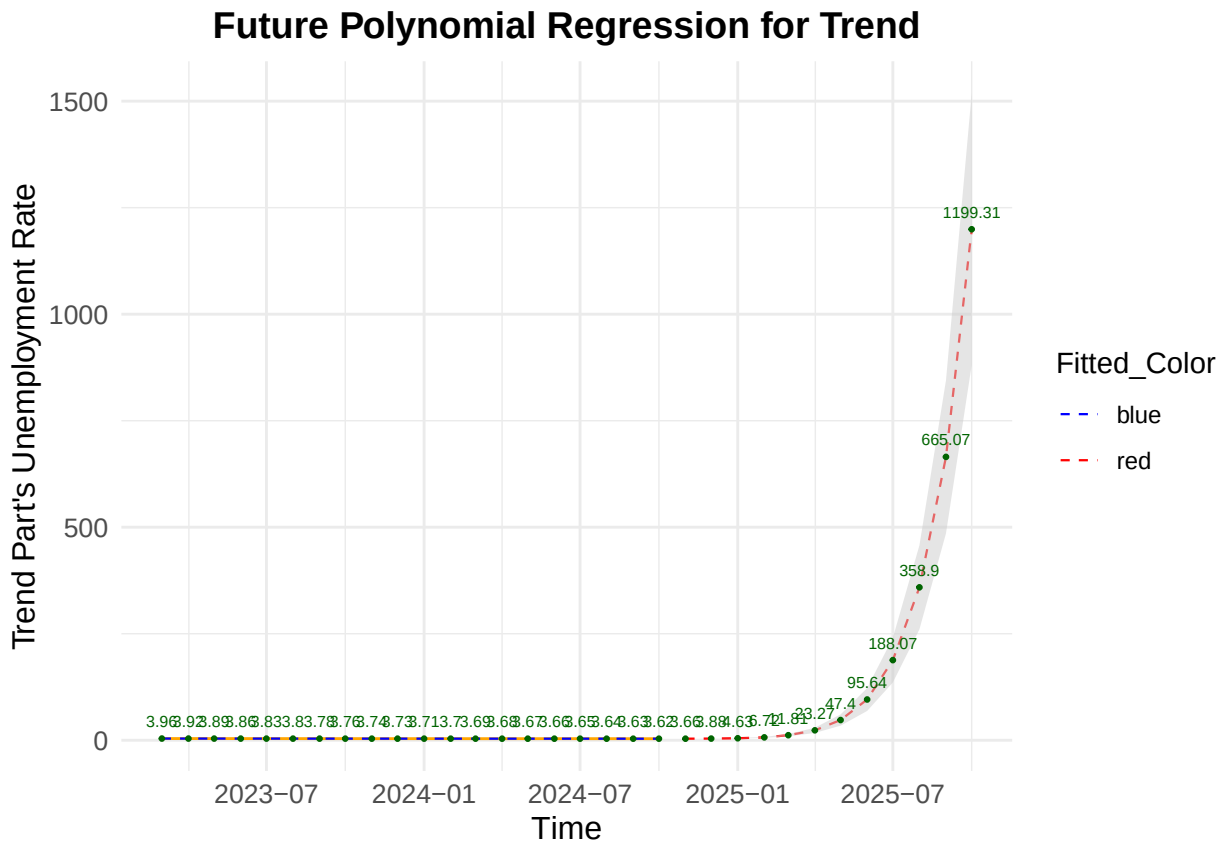
  # Display the fitted values as text labels on the plot
  geom_text(data = plot_data, aes(x = Time, y = Fitted, label = paste0(round(Fitted, 2))),
```

```
            vjust = -1, hjust = 0.5, color = "darkgreen", size = 2) +

  # Add labels and title to the plot
  labs(title = "Future Polynomial Regression for Trend",
       x = "Time",
       y = "Trend Part's Unemployment Rate") +

  # Apply minimal theme for the plot
  theme_minimal() +

  # Customize the appearance of the plot
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10)
  )
```



#Secrion 5

##(Table 21) Forecast for the next 12 months of the seasonally adjusted unemployment rate & (Figure 13) Longe term polynomial prediction

```
fit_arima <- auto.arima(ts_unrate)

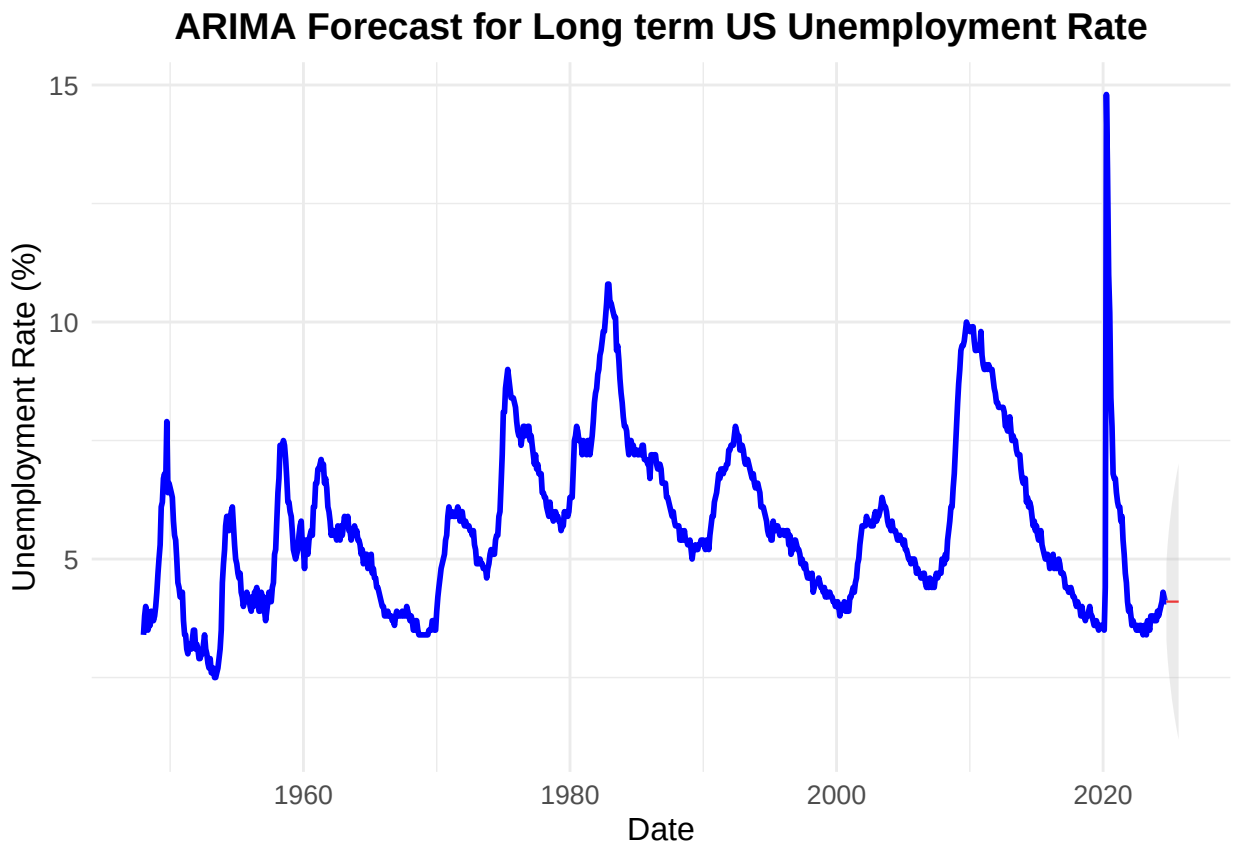
# Forecast the next 12 months
forecast_result <- forecast(fit_arima, h = 12)
```

```r
print(forecast_result)
```

```
##          Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## Nov 2024       4.102696 3.569280 4.636112 3.286907 4.918485
## Dec 2024       4.100603 3.324832 4.876374 2.914164 5.287042
## Jan 2025       4.102228 3.156963 5.047493 2.656570 5.547886
## Feb 2025       4.100967 3.003243 5.198690 2.422143 5.779790
## Mar 2025       4.101946 2.876765 5.327127 2.228193 5.975698
## Apr 2025       4.101186 2.756172 5.446199 2.044165 6.158206
## May 2025       4.101776 2.649957 5.553595 1.881410 6.322141
## Jun 2025       4.101318 2.547714 5.654921 1.725286 6.477349
## Jul 2025       4.101673 2.454250 5.749096 1.582158 6.621189
## Aug 2025       4.101397 2.363970 5.838824 1.444232 6.758562
## Sep 2025       4.101611 2.279542 5.923681 1.314997 6.888226
## Oct 2025       4.101445 2.197808 6.005082 1.190084 7.012806
```

```r
# Create a data frame for plotting
forecast_df <- data.frame(
  Date = seq.Date(from = max(data$DATE), by = "month", length.out = 12),
  Forecast = forecast_result$mean,
  Lower = forecast_result$lower[,2],  # 95% confidence interval
  Upper = forecast_result$upper[,2]   # 95% confidence interval
)

# Plot
ggplot() +
  geom_line(aes(x = data$DATE, y = ts_unrate), color = "blue", size = 1) +  # Actual values
  geom_line(aes(x = forecast_df$Date, y = forecast_df$Forecast), color = "red", linetype = "solid", siz
  geom_ribbon(aes(x = forecast_df$Date, ymin = forecast_df$Lower, ymax = forecast_df$Upper), fill = "gr
  labs(title = "ARIMA Forecast for Long term US Unemployment Rate",
       x = "Date", y = "Unemployment Rate (%)") +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10)
  )
```


ARIMA Forecast for Long term US Unemployment Rate
Unemployment Rate (%)
15
10
5
1960
1980
2000
2020
Date